# Descripción del flujo de practica ADO.NET

El proyecto esta separado en capas y cada una tiene una sola responsabilidad:

1. Base de Datos (Database/script.sql): Crea la base ProductosDB y la tabla Productos con sus restricciones (precio mayor a 0, stock no negativo). Tambien se le insertan algunos productos como ejemplos.

2. Modelo. (Models/Producto.cs): Es la clase que representa un producto. Sus propiedades corresponden a las columnas de la tabla. Contien validaciones ([Required], [Range], [StringLength]) que ayuda a evitar que entren datos incorrectos.

3- Conexión (Data/ConexionDb.cs): Lee la cadena de conexion de appsettings.json y entrega un objeto SqlConnection. cada vez que se necesita hablar con la base de datos.

4- Repositorio (Repositories/):
- IProductRepository: Es la interfaz que define que operaciones existen, pero no cómo se hacen.
- ProductRepository: La implementa y contiene codigo SQL que se ejecuta.

5- Controlador (Controllers/ProductosController.cs): Recibe las peticiones HTTP, llama al repositorio y regresa la respuesta con el codigo de estado adecuado.

6- Arranque (Program.cs): Configura la aplicacion, registra la conexion y el repositorio en la inyeccion de dependencias, activa los controladores y habilita Swagger.

Recorrido realizado cuando se recibe una petición.

1- El usuario hace una peticion desde Swagger, como por ejemplo GET/api/productos/2
2- ASP.NET Core busca la accion del controlador que corresponde a esa ruta y verbo HTTP.
3- Si la peticion trae datos (POST o PUT), primero validan con las reglas del modelo. Si algo sale mal, se responde 400 sin tocar la base de datos.
4- El controlador no se conecta a la base de datos por si mismo., Usa la interfaz IProductRepository, que la inyeccion de dependencias la entrega ya creada.
5- El repositorio pide una conexion a ConexionDB y arma un SqlCommand con la consulta y sus comandos. Luego abre la conexion y la ejecuta.
6- El resultado regresa al controlador. Este decide la respuesta: 200 si todo salio bien, 201 si se creo algo, 204 si se actualizo o elimino algo, 404 si el producto no existe.
7- La respuesta llega a Swagger en formato JSON.

## Metodos del repositorio

- ObtenerTodos() (Consultar): Ejecuta un SELECT de toda la tabla. Lee los resultados fila por fila con un SqlDataReader y convierte cada una en un objeto Producto. Regresa la lista completa.

- ObtenerPorId(int id) (Buscar): Hace un SELECT con WHERE Id = @Id. Si se encuentra la fila, regresa el producto; si no, regresa null, y el controlador lo convierte en un 404.

- Insertar(Producto producto) (Insert): Ejecuta un INSERT con los datos del producto. Con OUTPUT INSERTED.ID y ExecuteScalar() obtiene el Id que se genero en la base de datos y lo regresa. El Id y la fecha de registro los pone SQL Server de forma automatica.

- Actualizar(Producto producto) (Actualizar): Ejecuta un UPDATE sobre un producto con ese Id. ExecuteNonQuery() indica cuantas filas se ejecutaron. Si fue al menos una fila regresa un true; si no el producto no existia entonces regresa un false.

- Eliminar(int id) (Eliminar): Ejecuta un DELETE por Id. Al igual que actualizar, regresa un true si borro algo; sino regresa un false si el producto no existia.

## Relacion entre endpoints y métodos

| Endpoint | Métodos del repositorio |
|---|---|
| GET /api/productos | ObtenerTodos() |
| GET /api/productos/{id} | ObtenerPorId(id) |
| POST /api/productos | Insertar(producto) |
| PUT /api/productos/{id} | Actualizar(producto) |
| DELETE /api/productos/{id} | Eliminar(id) |